Only Copilot Fans

Agentic Development Framework

Simple Enough for Functional Consultants. Robust Enough for Pro Developers.

Created by AJ Ansari, Microsoft MVP

The Playbook — four phases, twelve steps, twelve gates

READING THE SCHEMATICS

Shapes and roles

SHAPE LEGEND

Start / end	Start or end point of a flow
Action	An action, step, or process
Doc	Input or output — data, a document
Exit gate	An exit gate — the checkpoint that must be met
Choice?	A decision point
All along	A cross-cutting concern, running underneath

ROLE LEGEND — §1.7

Main role
Generates and edits all code; owns the continuity documents.

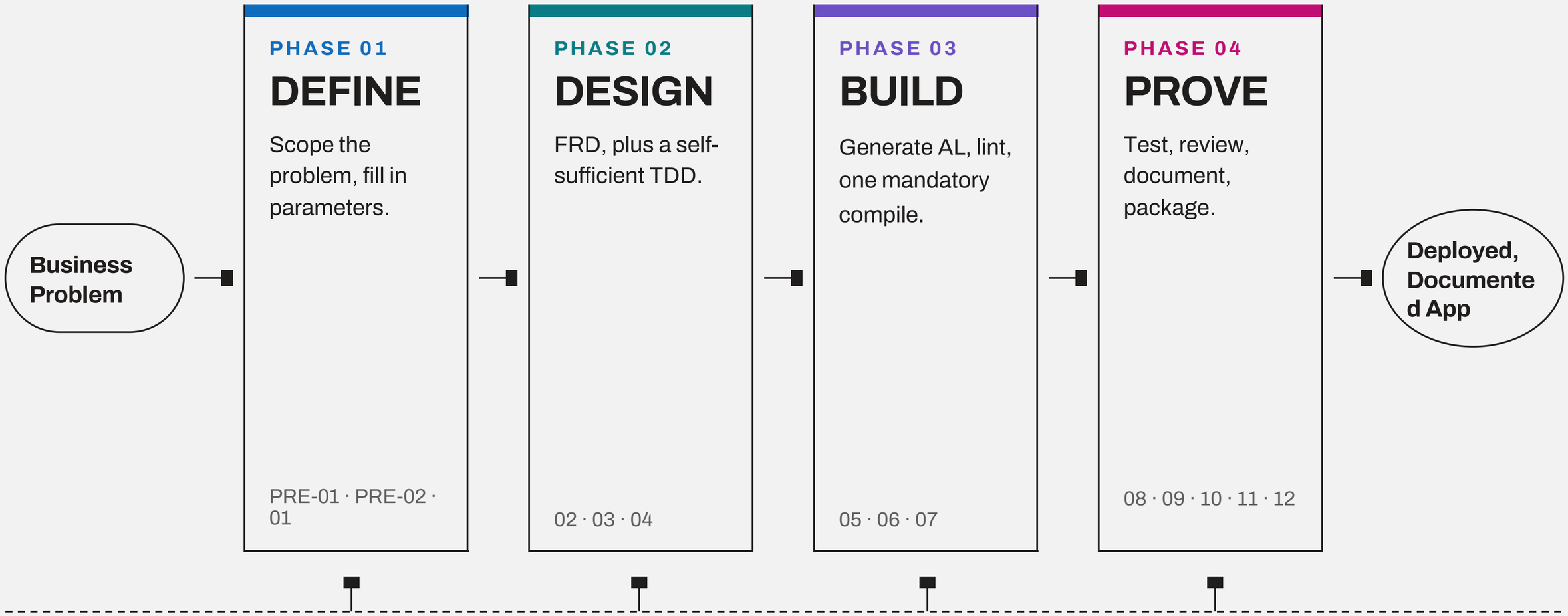
Light role
Post-generation pre-flight only. Reports findings, never edits.

Reasoning role
Authors, reviews, diagnoses. Never edits code or continuity docs.

Where §1.7 role assignment isn't configured, every colored step is done by the single running model instead.

SCHEMATIC 1

The whole framework in one picture



SCHEMATIC 2 — STEP AND GATE FLOW, PART ONE

Every step, and the gate that closes it

DEFINE

PRE-01 · State the Problem

Functional Consultant signs off

PRE-02 · Structured Gap Analysis

Technical Lead reviews; gaps closed or deferred

01 · Populate the Intake Sheet

No placeholders; human confirms sheet

↓ to DESIGN, Step 02

DESIGN

02 · Craft the FRD

Dev Manager sign-off

03 · Craft the TDD

Technical Lead sign-off; self-sufficiency check

04 · Sanity Check and Validation

0 blocking issues

↓ to BUILD, Step 05

SCHEMATIC 2 — STEP AND GATE FLOW, PART TWO

Every step, and the gate that closes it

BUILD

05 · Plan the Code

Batch order agreed; scaffold structurally complete

06 · Code Generation — per batch, no compile

Every batch pre-flight clean; permission-set coverage verified

07 · Troubleshoot, Iterate — the one compile

0 errors / 0 warnings

↓ to PROVE, Step 08

PROVE

08 · Gap-Fit Test

Every gap classified, resolved or scheduled

09 · Package and Test the App

Publishes cleanly; green/red-team pass

10 · Code Review

Critical findings resolved; dead-code scan clean

11 · Update Design Documents

As-built TDD complete; FRD reflects reality

12 · Document the Code

4 docs exist; ready for UAT

SCHEMATIC 3 — CROSS-CUTTING

Model & Effort Assignment (§1.7)

§1.7 configured?

No (default) — everything runs through one model, as if §1.7 didn't exist.

Yes — a fixed three-role division, regardless of which physical models are assigned.

MAIN ROLE

All BUILD code generation. All actual code edits, including applying what Light and Reasoning report. End-to-end ownership of the ChangeLog, Object Register, ProjectMemory and TestingFeedback.md.

LIGHT ROLE

The Step 05 post-generation pre-flight pass only — required props, Rec.-qualification, dead code, indentation, permission-set coverage, symbol verification.

Reports findings. Never edits code.

REASONING ROLE

Sanity Check (04), Gap-Fit Test (08), Code Review (10), FRD (02) and TDD (03) authorship, root-cause troubleshooting (07), diagnosing testing-feedback reports.

Reports drafts and diagnoses. Never edits code or continuity docs.

Light and Reasoning relay findings to Main

The codebase & continuity documents

PHASE 01

DEFINE

Scope the problem and fill in the parameters. Ask, don't infer.

PRE-01 State the Problem · PRE-02 Structured Gap Analysis · 01 Populate the Intake Sheet

PHASE 01 — DEFINE · SCHEMATIC 4.1

From stakeholder conversation to confirmed parameters

Stakeholder conversation — business need in plain language

PRE-01 · STATE THE PROBLEM

INPUTS	ACTIONS	OUTPUT
Stakeholder notes	Write the problem statement, capture vocabulary, draft the initial entity list, flag ambiguities.	ProblemStatement.md

EXIT GATE — Functional Consultant signs off

PRE-02 · STRUCTURED GAP ANALYSIS

INPUTS	ACTIONS	OUTPUT
ProblemStatement.md, initial entity list	Run the gap checklist — analytical, posted, lookup, secondary docs, legacy vs. modern, tax, global/local.	Expanded entity list + gap log

EXIT GATE — Technical Lead reviews; gaps closed or deferred

01 · POPULATE THE INTAKE SHEET

INPUTS	ACTIONS	OUTPUT
Expanded entity list, symbol file	§1.1 Identity · §1.2 ID ranges · §1.3 Naming/API · §1.4 Platform · §1.6 Onboarding & Discoverability · §1.7 Model & Effort Assignment — ask, don't infer.	Project Parameters, Object Register (seeded)

EXIT GATE — no placeholders remain; a human confirms the sheet → **to DESIGN, Step 02**

PHASE 02

DESIGN

An FRD in business language, and a TDD complete enough to build from alone.

The Main role integrates every draft — saves the file, keeps ChangeLog and ProjectMemory bookkeeping — and carries it to the human for sign-off. The Reasoning role never owns sign-off.

02 Craft the FRD · 03 Craft the TDD · 04 Sanity Check and Validation

PHASE 02 — DESIGN · SCHEMATIC 4.2

Write what, then how — and check they agree

02 · CRAFT THE FRD

REASONING ROLE

INPUTS
ProblemStatement.md, expanded entity list, Project Parameters

ACTIONS
Write in business language — what and why, not how. Validate against DEFINE.

OUTPUT
FRD.md

EXIT GATE — Dev Manager sign-off; every entity accounted for

03 · CRAFT THE TDD

REASONING ROLE

INPUTS
FRD.md, Project Parameters, symbol file

ACTIONS
System identity, module grouping, batch plan, per-object and per-field spec, permission sets — it must be self-sufficient.

OUTPUT
TDD.md, Object Register updated

EXIT GATE — Technical Lead sign-off; self-sufficiency check

04 · SANITY CHECK AND VALIDATION

REASONING ROLE

INPUTS
FRD.md, TDD.md, symbol file

ACTIONS
Formal review — can BC actually do this? Does the TDD implement the FRD? Work the checklist.

OUTPUT
SanityCheck.md

EXIT GATE — 0 blocking issues → **to BUILD, Step 05**

PHASE 03

BUILD

No per-batch compile. Symbol-verified lint instead, and exactly one mandatory compile once every planned batch exists.

05 Plan the Code · 06 Code Generation · 07 Troubleshoot, Iterate

PHASE 03 — BUILD · SCHEMATIC 4.3, STEPS 05–06

Plan the batches, then generate them — without compiling

05 · PLAN THE CODE

INPUTS
TDD.md, Object Register

ACTIONS
Confirm batch order, prepare the scaffold, bootstrap the AL MCP Server and BCQuality snapshot, write the pre-flight checks.

OUTPUT
Batch plan, scaffold, pre-flight checklist

EXIT GATE — batch order agreed; scaffold complete, not compiled

06 · CODE GENERATION

repeats once per batch

MAIN
Pause for human approval, then generate the batch's AL files from the standard template.

MAIN · PRE-GEN PRE-FLIGHT
Identifier length, reserved keywords, localization, ObsoleteState — on planned names. Fix the TDD if needed.

LIGHT · POST-GEN PRE-FLIGHT
Required props, Rec.-qualification, dead code, indentation, coverage, symbol verification. Reports; Main fixes.

LIGHT · COVERAGE
Verify permission-set coverage across all batches so far — design-time, independent of compiling.

More batches?

Yes — return to the top of the loop for the next batch

No — every planned batch generated

EXIT GATE — every batch pre-flight clean; coverage verified; no compile here

PHASE 03 — BUILD · SCHEMATIC 4.3, STEP 07

Troubleshoot, iterate — the one mandatory compile

FIRST: compile the whole extension once — the one mandatory compile (Operating Rule 4).

Errors or warnings?

No ↓

0 / 0
errors / warnings

Yes

REASONING
Diagnose: one-off or pattern? Where did it come from? What rule missed it?

MAIN
Fix the root cause, regenerate files, re-compile, log the ChangeLog and update the TDD.

back to compile — repeat until clean

EXIT GATE — full extension compiles clean; ChangeLog and TDD reconciled

PHASE 04

PROVE

Test, review, document, package — until it is ready for UAT.

08 Gap-Fit Test · 09 Package and Test · 10 Code Review · 11 Update Design Documents · 12 Document the Code

PHASE 04 — PROVE · SCHEMATIC 4.4, STEPS 08–09

Does the built thing match what was promised?

08 · GAP-FIT TEST, FIDELITY VALIDATION

REASONING ROLE

INPUTS
FRD.md, TDD.md, built AL, ChangeLog

ACTIONS
Three-way comparison — FRD vs. TDD vs. as-built. Classify every gap.

OUTPUT
GapAnalysis.md, gap-fill work items

EXIT GATE — every gap classified, resolved or scheduled

Gap-fill work?

Yes — each item takes its own pass through the Step 05/06/07 discipline: pre-flight, then compile (Operating Rule 4).

No — straight to Step 09

09 · PACKAGE AND TEST THE APP

INPUTS
Clean-compiling extension, permission sets if any tables

ACTIONS
Build the .app (name and version from app.json), publish to sandbox, run green-team and red-team tests, verify permission sets.

OUTPUT
.app package, test-run record

EXIT GATE — publishes cleanly; green tests pass; red tests fail gracefully

PHASE 04 — PROVE · SCHEMATIC 4.4, STEPS 10–12

Review it, then leave the documentation behind

10 · CODE REVIEW

REASONING ROLE

INPUTS
Full built extension, TDD.md, Standards

ACTIONS
Code quality, dead code, obsolete refs, anti-patterns, permission-set re-verify, BCQuality knowledge-backed review.

OUTPUT
CodeReview.md

EXIT GATE — critical findings resolved; dead-code scan 100% clean

11 · UPDATE DESIGN DOCUMENTS

INPUTS
TDD.md, FRD.md, ChangeLog, GapAnalysis.md, CodeReview.md

ACTIONS
Write a new PostDevTDD.md (as-built); update FRD.md in place as the new baseline.

OUTPUT
PostDevTDD.md, updated FRD.md

EXIT GATE — as-built TDD regenerable; FRD reflects reality

12 · DOCUMENT THE CODE

INPUTS
PostDevTDD.md, built AL, Step 09 test-run record

ACTIONS
Generate reference docs and a Mermaid ER diagram — render it — plus a human test script, UserGuide.md and Deployment.md.

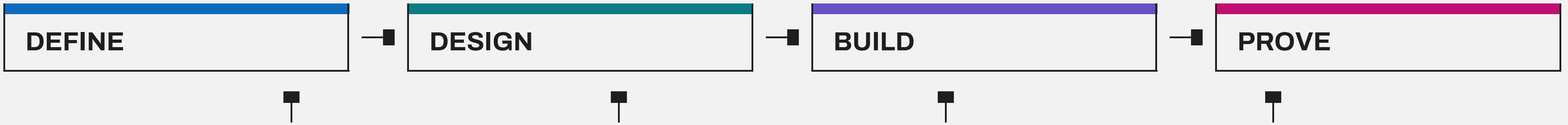
OUTPUT
Documentation · UserGuide · HumanUnitTestScript · Deployment

EXIT GATE — reference generated from code; ready for UAT

Deployed, Documented App

SCHEMATIC 5 — CROSS-CUTTING

ALL ALONG — continuous discipline



ALL ALONG

EVERY PHASE, EVERY STEP

Document

Keep every required doc current, not retroactively.

Track Changes

Every FRD/TDD deviation logged before the next batch.

Retain Explanations

Who decided, and why — not just the outcome.

Testing Feedback Log

Verbatim findings, triaged explicitly, cross-referenced.

Project Memory

docs/ProjectMemory.md — an anchor, not a narrative.

Packaging & Versioning

Never delete a package; propose bumps, don't apply silently.

AL MCP Server

Bootstrap once; prefer its tools over ad hoc terminal use.

BCQuality Snapshot

Fetch once; refresh only on explicit request.

ONLYCOPILOTFANS — AGENTIC DEVELOPMENT FRAMEWORK

Simple Enough for Functional Consultants. Robust Enough for Pro Developers.

Created by AJ Ansari, Microsoft MVP